Guía de Ramas en Git y Conventional Branch

Flujo de trabajo en equipo, comandos esenciales y convención de nombres

Guía práctica para el Trabajo Práctico de Desarrollo de Software

1. ¿Cómo funciona el trabajo con Ramas (Branches) en Git?

En un proyecto grupal, la rama principal (`main`) debe contener siempre código estable y que compile sin errores. Ningún integrante debe programar directamente sobre `main`. En su lugar, cada vez que alguien va a desarrollar un Caso de Uso, corregir un error o realizar una tarea, crea una **rama temporal aislada** a partir de `main`, trabaja allí y, al finalizar, solicita unir sus cambios mediante un **Pull Request (PR)** en GitHub.

Comandos esenciales de ramas y su explicación

- **Ver en qué rama estás parado y listar las ramas locales:**

  ```
  git branch
  ```

  Explicación: Muestra todas las ramas descargadas en tu computadora y marca con un asterisco (`*`) la rama activa en la que estás trabajando actualmente.

- **Cambiar a una rama existente:**

  ```
  git checkout <nombre-rama>
  ```

  Explicación: Mueve tu espacio de trabajo a la rama indicada (por ejemplo, `git checkout main`).

- **Crear una rama nueva y moverte a ella en un solo paso:**

  ```
  git checkout -b <tipo>/<descripcion>
  ```

  Explicación: Crea una nueva rama copiando el estado exacto de la rama donde estabas parado y te cambia automáticamente a ella para empezar a programar.

- **Descargar información de ramas nuevas desde GitHub:**

  ```
  git fetch origin
  ```

  Explicación: Consulta al servidor remoto (`origin`, es decir, GitHub) si algún compañero creó ramas nuevas o subió cambios, actualizando las referencias sin modificar tus archivos todavía.

- **Traer y fusionar los últimos cambios de una rama remota:**

  ```
  git pull origin <nombre-rama>
  ```

  Explicación: Descarga los últimos commits desde GitHub para esa rama y los aplica en tu computadora.

- **Subir una rama nueva a GitHub por primera vez:**

  ```
  git push -u origin <nombre-rama>
  ```

  Explicación: Publica tu rama local en GitHub. El parámetro `-u` vincula tu rama local con la remota para que en los siguientes envíos solo necesites escribir `git push`.

2. Convención de Nombres: Conventional Branch

Conventional Branch es una convención estandarizada para nombrar ramas en Git de manera legible y ordenada. A excepción de las ramas troncales (`main`, `master`, `develop`), toda rama de trabajo debe seguir la estructura:

```
<type>/<description>
```

Prefijos utilizados en el proyecto

1. `feature/` (o su alias corto `feat/`): Se utiliza para desarrollar **nuevas funcionalidades** o Casos de Uso del sistema.
   - *Ejemplos:* `feature/cu01-autenticar-usuario`, `feature/cu09-alta-puesto`
2. `bugfix/` (o su alias corto `fix/`): Se utiliza para **corregir errores (*bugs*)** detectados durante el desarrollo o las pruebas que no revisten urgencia crítica inmediata.
   - *Ejemplos:* `bugfix/validacion-puntaje-negativo`, `fix/error-carga-preguntas`
3. `hotfix/`: Se utiliza para **correcciones urgentes o críticas** que impiden el funcionamiento del sistema en la rama principal o justo antes de una entrega.
   - *Ejemplos:* `hotfix/conexion-base-de-datos`, `hotfix/error-compilacion-api`
4. `core/` (o `chore/` en la especificación estándar): Se utiliza para tareas estructurales, de configuración base, mantenimiento, actualización de dependencias, scripts SQL o documentación que no corresponden a un Caso de Uso funcional.
   - *Ejemplos:* `core/config-spring`, `core/script-tablas`, `core/actualizar-readme`

Reglas básicas de escritura para el nombre de la rama

- **Solo minúsculas, números y guiones medios:** Usar únicamente letras minúsculas (`a-z`), números (`0-9`) y guiones medios (`-`) para separar palabras. **No usar** mayúsculas, espacios, acentos ni guiones bajos (`_`). Se permiten puntos (`.`) solo para números de versión (ej. `v1.2`).
- **Sin guiones ni puntos consecutivos o en los extremos:** No escribir `feature/alta-puesto`, `feature/-alta-puesto` ni `feature/alta-puesto-`.
- **Nombres claros y concisos:** El nombre debe describir brevemente qué objetivo cumple esa rama.

Nombre de la Rama	¿Válido?	Motivo
`main`	Sí	Rama principal troncal (no lleva prefijo).
`feature/cu09-alta-puesto`	Sí	Nueva funcionalidad con formato correcto.
`bugfix/corregir-filtro-candidatos`	Sí	Corrección de un bug en el código.
`hotfix/arreglar-login-caido`	Sí	Parche crítico urgente.
`core/configurar-conexion-postgres`	Sí	Tarea base/estructural o de configuración.
`Feature/Alta_Puesto`	**No**	Contiene mayúsculas y guión bajo.
`feature/alta-puesto-`	**No**	Tiene guiones consecutivos y termina en guión.

3. Proceso Paso a Paso: Del Código al Pull Request

A continuación se detalla el ciclo completo que debe seguir cada integrante al trabajar en una tarea:

Paso 1: Crear una rama nueva (o sumarse a una ya existente)

Caso A: Vas a empezar una tarea nueva desde cero. Siempre debes partir desde la rama `main` actualizada para no trabajar sobre código viejo:

```
git checkout main
git pull origin main
git checkout -b feature/cu09-alta-puesto
```

Caso B: La rama ya fue creada por un compañero en GitHub y quieres trabajar en ella:

```
git fetch origin
git checkout feature/cu09-alta-puesto
git pull origin feature/cu09-alta-puesto
```

Paso 2: Trabajar y guardar avances (Commits)

Realizas las modificaciones en el código y vas registrando tus avances usando *Conventional Commits*:

```
git status
git add .
git commit -m "feat(puestos):  implementar servicio y dao para alta de puesto"
```

Paso 3: Actualizar tu rama con los últimos cambios de `main`

Mientras tú trabajabas en tu rama, es probable que otro compañero haya terminado su tarea y haya actualizado `main`. **Antes de subir y pedir el Pull Request**, debes traer los cambios de `main` hacia tu propia rama para verificar que todo funcione en conjunto:

```
git fetch origin
git merge origin/main
```

Si nadie modificó las mismas líneas que tú, Git unirá los cambios automáticamente.

Paso 4: ¿Cómo resolver conflictos de fusión (Merge Problems)?

Si al ejecutar `git merge origin/main` Git te avisa que hay un **CONFLICT** (porque tú y un compañero editaron las mismas líneas del mismo archivo):

1. Ejecuta `git status` para ver qué archivos están marcados como *both modified* (en conflicto).
2. Abre esos archivos en tu editor (VS Code / IntelliJ). Verás marcas como estas:

   ```
   <<<<< HEAD
   // Tu código en tu rama actual
   =======
   // El código nuevo que vino desde main
   >>>>> origin/main
   ```

3. Edita el archivo dejando el código correcto (borrando los marcadores `<<<<<`, `=======` y `>>>>>`) o usando los botones del IDE (*Accept Current / Incoming / Both*).
4. Verifica que el proyecto compile bien y cierra la fusión con un commit:

   ```
   git add <archivo-resuelto>
   git commit -m "chore:  resolver conflictos de merge con main"
   ```

Paso 5: Pushear tu rama a GitHub

Una vez que tu rama tiene tu trabajo listo y está al día con `main` sin conflictos, súbela al repositorio remoto:

```
git push -u origin feature/cu09-alta-puesto
```

(Si ya habías hecho push antes en esta misma rama, basta con ejecutar `git push`).

Paso 6: Crear el Pull Request (PR) en GitHub

1. Entra al repositorio del grupo en **GitHub**. Verás un cartel amarillo sugiriendo **"Compare & pull request"** para tu rama recién subida (o ve a la pestaña *Pull requests* → *New pull request*).
2. Asegúrate de que las ramas seleccionadas sean:

   `base: main` ⟵ `compare: feature/cu09-alta-puesto`
3. Escribe un título claro (idealmente siguiendo Conventional Commits, por ejemplo: `feat(puestos): implementar CU 09 Dar de alta puesto`) y una breve descripción de lo que hiciste.
4. Haz clic en **"Create pull request"**.
5. Otro integrante del grupo revisará el código en GitHub y, si todo está correcto, hará clic en **"Merge pull request"** para integrar oficialmente tus cambios a `main` y luego en **"Delete branch"** para limpiar la rama temporal.